

GRCPC 2025 Editorial



A. zerorush

Author:

Analysis: Anastasis Koliopoulos.

Tags: Greedy.

To begin, we reduce the problem to a decrease-only scenario by considering the absolute values of the sequence. Next, we observe that all operations must be applied to a single element — specifically, the smallest even number. This choice is optimal, since it will reach zero in the fewest steps among all even numbers. Note that an odd number can never become zero, so a sequence consisting entirely of odd numbers admits no solution. Let a denote the selected number, and let k be the number of operations needed for a to reach zero. Then

$$a - 2k = 0 \iff a = 2k \iff k = \frac{a}{2},$$

which is the answer to the problem.

The above approach requires only a single pass through the sequence, yielding a time complexity of $\mathcal{O}(N)$.

Indicative code

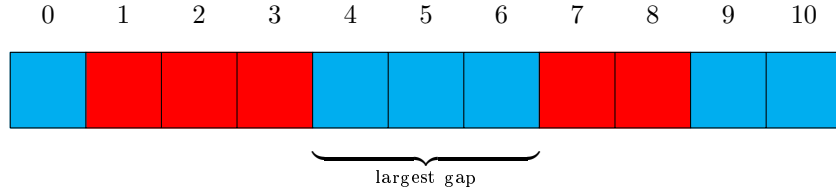
B. foodbreak

Author:

Analysis: Anastasis Koliopoulos.

Tags: Brute force, Sortings.

Let us first consider a simple, non-optimal but intuitive approach. We define an array $A[0 \dots T]$, where $A[i] = \text{red}$ if there is a presentation at timestamp i , and $A[i] = \text{blue}$ otherwise. For each presentation $i \in [N]$, we set every cell in the segment $[s_i, e_i) = [s_i, e_i - 1]$ to red. After processing all presentations, we find the longest contiguous segment consisting of cells with blue color, which gives us the answer.



This algorithm runs in worst-case time $\mathcal{O}(N \cdot \max_{1 \leq i \leq n} \{e_i - s_i\} + T) = \mathcal{O}(N \cdot T + T) = \mathcal{O}(N \cdot T)$, and requires $\mathcal{O}(T)$ space, which is insufficient to meet the time and memory limits. Let us now consider a better approach.

We consider the set of intervals $S = \{[s_i, e_i) : i \in [N]\} \cup \{(-\infty, 0), [T, \infty)\}$. For convenience, we will refer to the two additional intervals as (s_0, e_0) and $[s_{N+1}, e_{N+1})$, respectively.

We know that the desired interval must end exactly before the start of some presentation i and begin exactly after the end of some presentation j . That is, it must be of the form $[e_j, s_i) = [e_j, s_i - 1]$.

Observation. For a given interval i , where $1 \leq i \leq N + 1$, the maximum gap that ends before the start of i is equal to

$$\begin{aligned} \ell_i &= |[e_j, s_i - 1]| \\ &= \max\{s_i - 1 - e_j + 1, 0\} & ([e_j, s_i - 1] \subset \mathbb{N}) \\ &= \max\{s_i - e_j, 0\}, \end{aligned}$$

where $j = \arg \max_{0 \leq k \leq N} \{e_k : s_k < s_i\}$.

Proof. Suppose that there exists an interval j' such that $s_{j'} < s_i$ and $e_{j'} < e_j$, so that pairing interval i with j' would produce a larger gap than pairing i with j .

If $e_{j'} \geq s_i$, then no gap interval ending at time $s_i - 1$ is formed. Otherwise, if

$e_{j'} < s_i$, we consider two subcases. If $e_j \geq s_i$, this configuration does not yield any gap interval that ends at time $s_i - 1$. If not, e_j lies between $e_{j'}$ and s_i , and thus $[e_{j'}, s_i - 1]$ is not a gap.

■

Based on the above observations, we apply the following algorithm. We first sort the intervals by their left endpoint. We then scan them while maintaining the maximum right endpoint seen so far, denoted by $e_{\max}$. For every $i \in \{1, 2, \dots, N + 1\}$, we compute ℓ_i and update $e_{\max}$ accordingly. The final answer is given by $\max_{1 \leq i \leq N+1} \{\ell_i\}$.

The overall time complexity is $\mathcal{O}(N \cdot \log N)$ for sorting the intervals, followed by $\mathcal{O}(N)$ for the linear scan. Hence, the total complexity is dominated by the sorting step and is $\mathcal{O}(N \cdot \log N)$.

Indicative code

C. mansion

Author:

Analysis: Anastasis Koliopoulos.

Tags: Graphs, SSSP, Dijkstra's algorithm, 0/1 BFS.

We model the problem as follows. Let $G = (V, E)$ be a directed graph with $|V| = N$ and $|E| = M$, where each edge $e \in E$ has weight $w(e) \in \{0, 1\}$. The goal is to find the shortest path from vertex 1 to vertex N , or determine that no such path exists.

Since all edge weights are non-negative, we can apply *Dijkstra's algorithm* to compute the shortest path from vertex 1 to vertex N . Implementing Dijkstra's algorithm using an *std::set* or an *std::priority_queue* yields a time complexity of $\mathcal{O}((N + M) \cdot \log N)$, which is sufficient to pass all the test cases. Using a *Fibonacci heap* we can reduce the time complexity to $\mathcal{O}(N \cdot \log N + M)$, but its complex implementation and its large constant factor makes it impractical for competitive programming contests.

Actually, since all edge weights are either 0 or 1, we can instead use the more efficient *0/1 BFS* algorithm, which avoids the logarithmic factor and runs in $\mathcal{O}(N + M)$ time.

Indicative code

D. packages

Author:

Analysis: Anastasis Koliopoulos.

Tags: Binary search, k -th element.

We define the function $c : \{0, 1, \dots, n\} \rightarrow \mathbb{N}$, where $c(k)$ denotes the minimum cost to deliver k packages. We make the following crucial observation.

Observation. The cost function is non-decreasing.

Proof. We'll show that $c(k) \leq c(k+1)$, where $0 \leq k \leq n-1$. We denote by I_{k+1} the set of indices of the packages selected in an optimal solution with $k+1$ packages. For every $i \in I_{k+1}$ we have

$$\begin{aligned} k < k+1 &\implies k \cdot B[i] \leq (k+1) \cdot B[i] && (B[i] \geq 0) \\ &\implies A[i] + k \cdot B[i] \leq A[i] + (k+1) \cdot B[i]. \end{aligned}$$

Writing the above inequality for every i , we get

$$\left. \begin{array}{rcl} A[i_1] + k \cdot B[i_1] & \leq & A[i_1] + (k+1) \cdot B[i_1] \\ A[i_2] + k \cdot B[i_2] & \leq & A[i_2] + (k+1) \cdot B[i_2] \\ \vdots & & \\ A[i_{k+1}] + k \cdot B[i_{k+1}] & \leq & A[i_{k+1}] + (k+1) \cdot B[i_{k+1}] \end{array} \right\} \xRightarrow{+} \sum_{i \in I_{k+1}} A[i] + k \cdot B[i] \leq c(k+1).$$

Subtracting $A[j] + k \cdot B[j]$ from the left-hand side, for some $j \in I_{k+1}$, we obtain

$$\underbrace{\sum_{i \in I_{k+1} \setminus \{j\}} A[i] + k \cdot B[i]}_{\text{a feasible } k\text{-packages solution}} \geq c(k).$$

Combining the two inequalities above, we get

$$c(k) \leq c(k+1),$$

which completes the proof. ■

Leveraging the monotonicity of the cost function, we can *binary search the answer*. We choose a value $k \in \{0, 1, \dots, n\}$ and compute $c(k)$. If $c(k) \leq X$, where X denotes the available budget, we continue searching among values $k' \geq k$; otherwise, we restrict the search to values $k' < k$.

To determine which packages to deliver for a fixed k , we create the array $C[1 \dots n]$, where

$$C[i] = A[i] + k \cdot B[i]$$

and retrieve the k cheapest packages. There are three ways to obtain the desired packages, described below:

- Sort the array C in $\mathcal{O}(N \cdot \log N)$ time.
- Use the built-in C++ function `nth_element`, which runs in $\mathcal{O}(N)$ time on average.
- Use the *Median of medians algorithm*, which guarantees $\mathcal{O}(N)$ time in the worst case.

The binary search contributes an additional $\mathcal{O}(\log N)$ factor.

Overall, the best achievable time complexity is $\mathcal{O}(N \cdot \log N)$ in the worst case. Nevertheless, all of the above approaches are efficient enough to pass the given constraints. In the provided implementation, we use the second approach.

Indicative code

E. javelins

Author:

Analysis: Anastasis Koliopoulos.

Tags: Exchange arguments, DP, State rotation, Space optimization.

We will apply *Dynamic Programming (DP)* to solve this problem.

Before discussing the DP solution, observe that the order in which the targets are hit matters. Therefore, it is not sufficient to decide only which subset of targets to select, but also in which order to hit them.

Let I , where $I \subseteq [n]$ and $|I| \geq 2$, denote the set of the selected targets. Suppose we select two targets $i, i' \in I$. When is it optimal to hit target i before target i' ?

Let j, j' , with $1 \leq j < j' \leq |I|$, denote two time slots corresponding to the order in which the selected targets will be hit.

If we first assign target i to slot j and then target i' to slot j' , the energy consumption is given by

$$E = A[i] + (j - 1) \cdot B[i] + A[i'] + (j' - 1) \cdot B[i'].$$

If we swap the order in which we hit these two targets, the energy consumption is equal to

$$E' = A[i] + (j' - 1) \cdot B[i] + A[i'] + (j - 1) \cdot B[i'].$$

Solving the inequality below, we get

$$\begin{aligned} E < E' &\iff (j - 1) \cdot B[i] + (j' - 1) \cdot B[i'] < (j' - 1) \cdot B[i] + (j - 1) \cdot B[i'] \\ &\iff (j - 1) \cdot B[i] - (j' - 1) \cdot B[i] < (j - 1) \cdot B[i'] - (j' - 1) \cdot B[i'] \\ &\iff (j - j') \cdot B[i] < (j - j') \cdot B[i'] \\ &\iff B[i] > B[i']. \end{aligned}$$

This means that we hit target i before i' if and only if $B[i] > B[i']$.

Let's now determine the optimal set of targets using DP, after we sort the targets in decreasing order of $B[i]$. The five stages of dynamic programming are specified as follows.

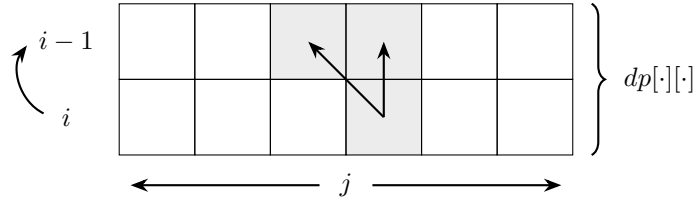
- (Definition of the DP table). We define $dp[i][j]$, where $0 \leq i, j \leq N$, as the minimum energy required to hit j out of the first i targets.
- (Choices). For a target i , we decide whether to skip this target or to include it as the j -th hit target.

- (Recurrence relation). The following recurrence is derived from the above choices:

$$dp[i][j] = \begin{cases} 0, & i = 0 \text{ and } j = 0, \\ \infty, & i = 0 \text{ and } 1 \leq j \leq N, \\ dp[i-1][0], & 1 \leq i \leq N \text{ and } j = 0, \\ \min \begin{cases} dp[i-1][j], \\ dp[i-1][j-1] + A[i] + B[i] \cdot (j-1), \end{cases} & 1 \leq i \leq N \text{ and } 1 \leq j \leq N. \end{cases}$$

- (Order of computation). The values are computed in increasing order of i .
- (Original problem solution). The solution is $\max_{0 \leq j \leq N} \{j : dp[N][j] \leq E\}$.

The final observation is that, in order to compute any DP value in row i , we only need to refer to row $i-1$. Therefore, at any step, instead of storing the entire DP table in memory, we can keep only the current and the previous row.



The time complexity of computing this DP recurrence is $\mathcal{O}(N^2)$, and, after applying the space optimization described above, the space complexity becomes $\mathcal{O}(N)$.

Note. Since the problem is formulated in terms of maximizing the answer, it is more intuitive for the DP recurrence relation to follow the same formulation. In fact, we could formulate a maximization-based recurrence relation as well; however, it would lead to a higher time complexity than the current approach. To avoid this, we applied *state rotation*.

Indicative code

F. audit

Author:

Analysis: Anastasis Koliopoulos.

Tags: Offline queries, Sparse Segment Tree.

Let's look at a simpler problem first. The statement is as follows.

Maintain a multiset of non-negative integers, denoted by S , that supports the following operations.

- $\text{INSERT}(S, a)$: Inserts the integer a into S .
- $\text{REMOVE}(S, a)$: Removes exactly one occurrence of a from S . It is guaranteed that a appears at least once in S .
- $\text{K-TH}(S, k)$: Returns the k -th smallest element of S . It is guaranteed that $1 \leq k \leq |S|$.

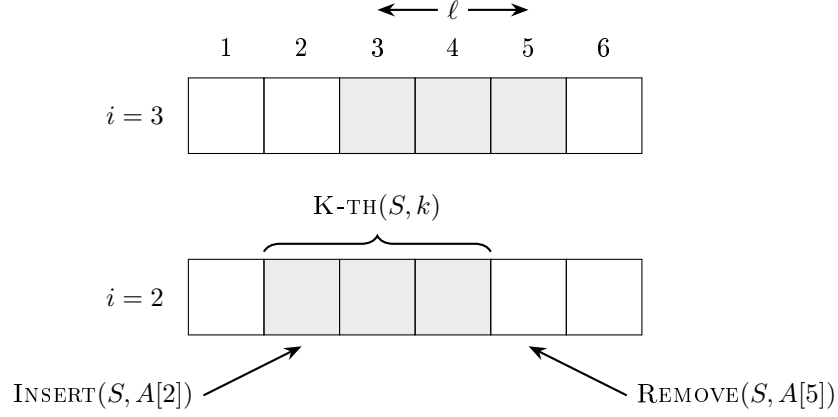
To handle these operations efficiently, we will use a *Sparse Segment Tree*. The structure of the tree and the INSERT operation are illustrated here. For the REMOVE operation, instead of adding 1 to each node along the path to the corresponding value, we simply subtract 1. Finally, for the K-TH operation, we can find the value corresponding to the k -th 1 using the method described here, commonly known as *Walking the Segment Tree*.

All of these operations traverse a path from the root to a leaf in our Segment Tree, performing a constant number of operations at each level. Suppose that the maximum value appearing in the tree is u . Then, the maximum depth of a leaf is $\lceil \lg u \rceil$. Therefore, each operation takes $\mathcal{O}(\lg u)$ time, giving a total complexity of $\mathcal{O}(Q \cdot \lg u)$ for Q operations.

Let's now return to our original problem. For a fixed query length ℓ , we traverse the array from right to left while maintaining the set S . When we reach an index i , we remove $A[i + \ell]$ from S , if $i + \ell \leq N$, and insert $A[i]$ into S . For every query whose range starts at i , we can simply call:

- $\text{K-TH}(S, 1)$ to find the minimum,
- $\text{K-TH}(S, k)$ to find the k -th smallest element, and
- $\text{K-TH}(S, \ell)$ to find the maximum.

Observe that S always contains the elements of a *sliding window* of size ℓ , as illustrated in the figure below.



We can map each query to its starting index during a preprocessing step. Since there are only two distinct window lengths, we can run the algorithm described above twice, once for each window length.

Reading the input array takes $\mathcal{O}(N)$ time, while reading the queries and performing the preprocessing step takes $\mathcal{O}(Q)$ time. Running the algorithm for both window lengths takes $\mathcal{O}(N \cdot \lg \max_i \{a_i\})$ time, since in this case $u = \max_i \{a_i\}$. Thus, the total time complexity is $\mathcal{O}(N \cdot \lg \max_i \{a_i\} + Q)$.

Note. Our algorithm is *offline*, meaning that we know all the queries in advance, allowing us to preprocess the data and answer the queries in any order. However, if we had to answer the queries *online* and for arbitrary ranges $[\ell, r]$, with the length of the range not fixed, we would need to use more sophisticated data structures, like *Persistent Segment Trees* or *Wavelet Trees*.

Indicative code